



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Software Engineering

Francesco Guerra
francesco.guerra@unimore.it

Objectives

- ▶ Understand the rationale behind the need to establish a discipline in software engineering.
- ▶ Analyze the main causes of software project failures.
- ▶ Give an example of software product failures.
- ▶ Understand the term software engineering
- ▶ Define the concepts of software engineering and of professionalism.
- ▶ Review the software engineering code of ethics.
- ▶ Discuss the sets of principles and foundations of software engineering
- ▶ Introduce the generic concept of software engineering process models

8-inch floppy disk



Examples and Characteristics of Software Failures

- ▶ <http://www.gao.gov/products/GAO-16-696T> (25 May 2016)

		in assembly language code and operates on an IBM mainframe.		Established for this update.
Department of Defense	Strategic Automated Command and Control System	Coordinates the operational functions of the United States' nuclear forces, such as intercontinental ballistic missiles, nuclear bombers, and tanker support aircrafts. This system runs on an IBM Series/1 Computer—a 1970s computing system—and uses 8-inch floppy disks.	53	Yes - The agency plans to update its data storage solutions, port expansion processors, portable terminals, and desktop terminals by the end of fiscal year 2017.
		Automates time and attendance for employees, timekeepers, payroll, and		Yes - The agency plans to

- ▶ As size and complexity of software projects increased, so did the number of failed projects
- ▶ “Engineering” Software was thought to be the cure:
 - ▶ Put some discipline into “programming”!
 - ▶ Do more than just coding/programming!
 - ▶ ‘Study’ (model/measure), ‘Understand’ (analyze), and ‘Improve’ (change) this field!

Project Failures

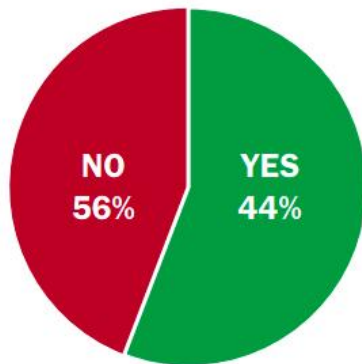
- ▶ Chaos Report (1995) by the Standish Group
 - ▶ Suggests that many of the mistakes in software projects are not well investigated
 - ▶ Some mistakes are continuously repeated
 - ▶ Sampled 300 software projects
 - ▶ Large, medium and small companies (banking, manufacturing, retail, ...)
 - ▶ and reported that **only about 16%** of those projects “completed,” “on-time,” and “within-budget”
 - ▶ 84% of projects failed!
- ▶ Chaos Report (2009) stated that software projects have improved with 32% “completed,” “on-time,” and “within-budget.”
 - ▶ That is still 68% of projects-failure!

Definition of Success

- ▶ The Chaos Report uses three measures of project success:
 - ▶ Did it deliver on time
 - ▶ Did it deliver to budget
 - ▶ Did it deliver an outcome that was satisfactory to the customer
- ▶ then a project outcome can be:
 - ▶ Successful - the project met all three measures of success
 - ▶ Challenged - the project did not meet all three measures of success
 - ▶ Failed - the project was cancelled or the product was never used

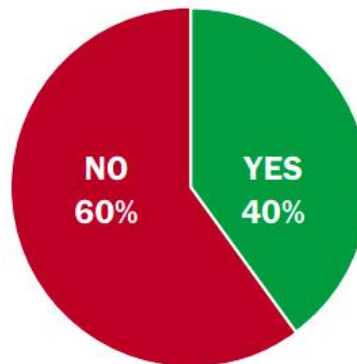
Project Failures (2)

ONBUDGET



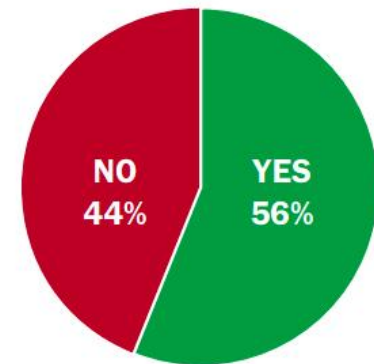
The percentage of projects that were OnBudget from FY2011–2015 within the new CHAOS database.

ONTIME



The percentage of projects that were OnTime from FY2011–2015 within the new CHAOS database.

ONTARGET



The percentage of projects that were OnTarget from FY2011–2015 within the new CHAOS database.

CHAOS Report 2015



Project Failures (3)

- ▶ Some important reasons for project success are
 - ▶ User involvement / Executive management support
 - ▶ Clear requirement statements
 - ▶ Proper planning
- ▶ The top failure factors for challenged projects
 - ▶ Lack of user input
 - ▶ Incomplete requirements and specifications
 - ▶ Changing requirements and specifications
- ▶ The top failure factors for impaired or cancelled projects are
 - ▶ Incomplete requirements
 - ▶ Lack of user involvement
 - ▶ Lack of resources

Project Failures (4)

- ▶ User involvement and user requirements are key success factors
 - ▶ Without understanding what is to be developed, there is a little chance of success for any project
 - ▶ Software projects are difficult to specify because many aspects of the work involved are nontangible

Project Failure (5)

CHAOS RESOLUTION BY AGILE VERSUS WATERFALL

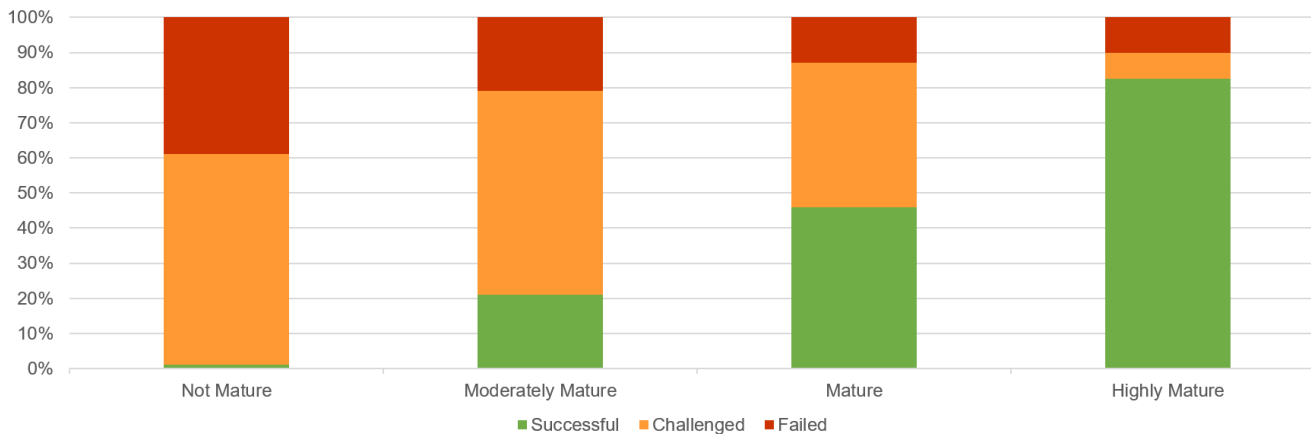
SIZE	METHOD	SUCCESSFUL	CHALLENGED	FAILED
All Size Projects	Agile	39%	52%	9%
	Waterfall	11%	60%	29%
Large Size Projects	Agile	18%	59%	23%
	Waterfall	3%	55%	42%
Medium Size Projects	Agile	27%	62%	11%
	Waterfall	7%	68%	25%
Small Size Projects	Agile	58%	38%	4%
	Waterfall	44%	45%	11%

The resolution of all software projects from FY2011–2015 within the new CHAOS database, segmented by the agile process and waterfall method. The total number of software projects is over 10,000.



CHAOS Report 2020

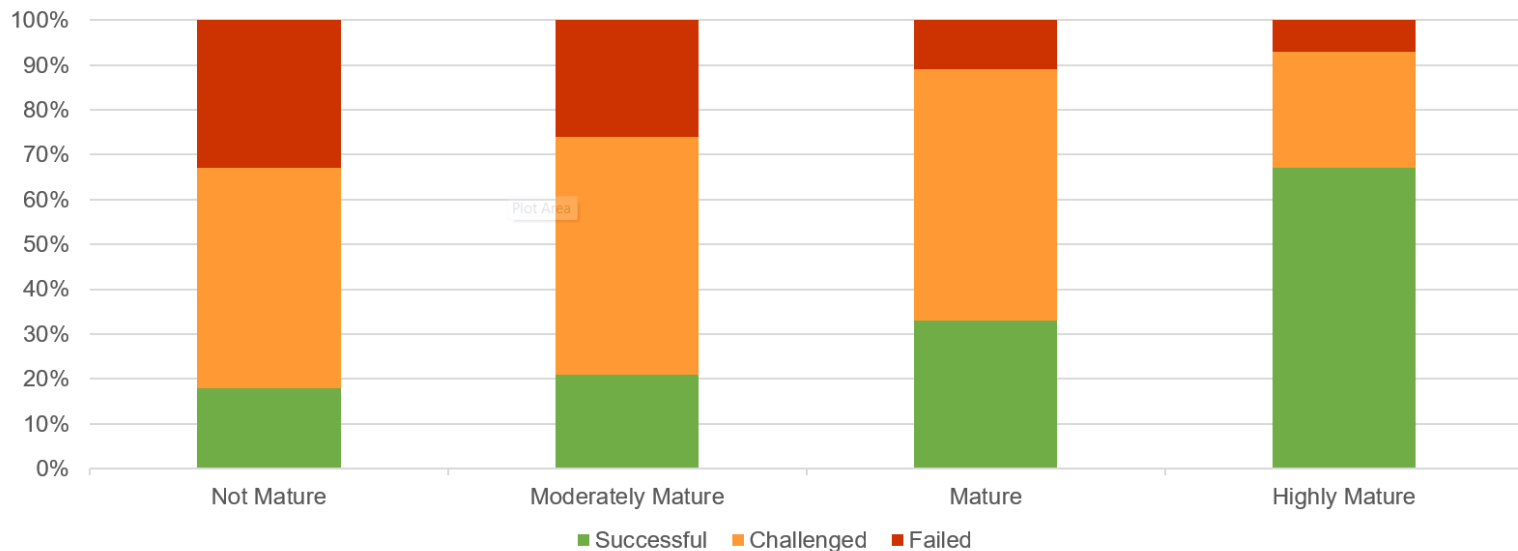
- ▶ 1- people make projects successful
 - ▶ A project is more likely to succeed if it had a strong team, an engaged executive sponsor, and operated within a supportive organizational culture.
 - ▶ The good team



- ▶ influence without authority; mindfulness; problem solving; communication; acceptance; respect; constructive confrontation; civility; determination/drive; emotional maturity.

CHAOS Report 2020

► The good sponsor

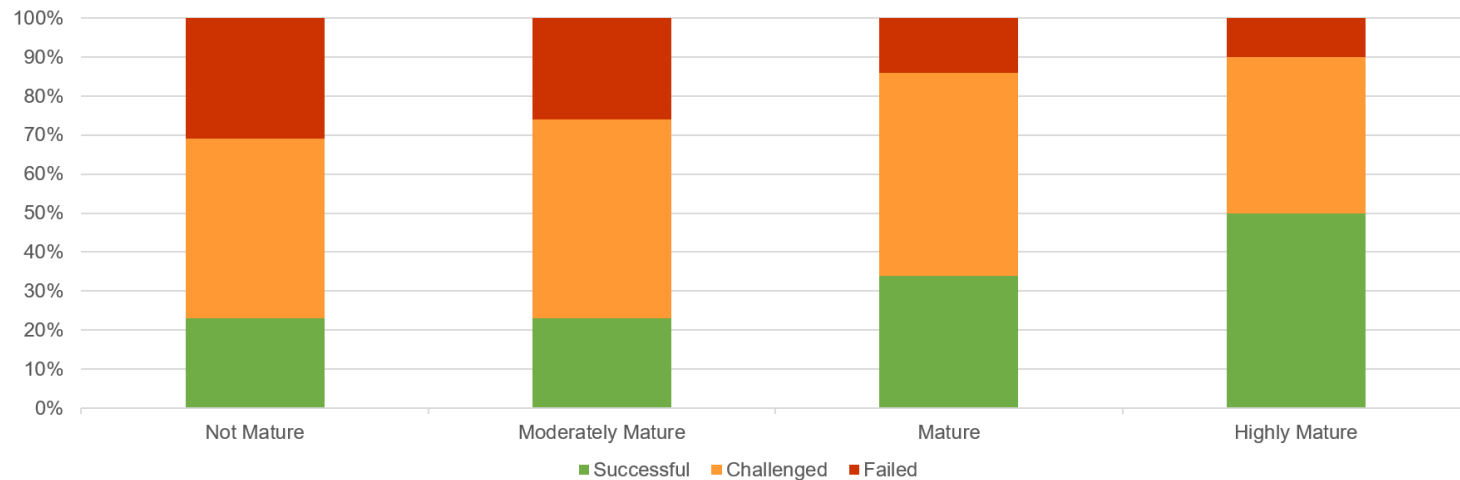


- decision latency; clear vision; influence; communication; active engagement; availability.

►

CHAOS Report 2020

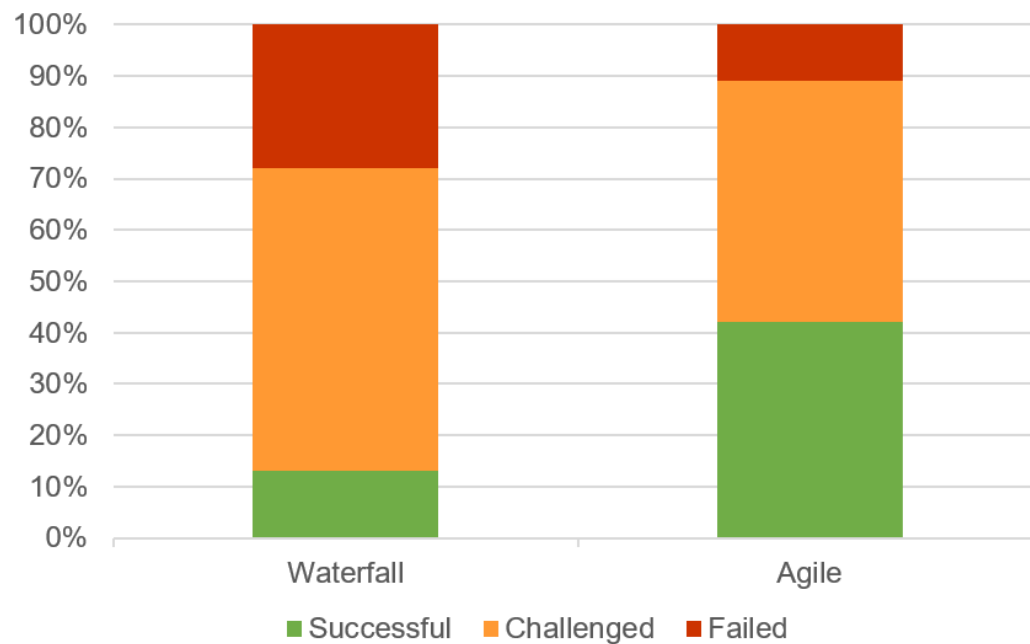
► The good place



- user involvement; emotional maturity; optimization; fast execution.
- The “Good Place” is the broader environment around the sponsor and team—the support functions and stakeholders that can either enable or hinder delivery—so improving their skills and behaviors is essential for project success.

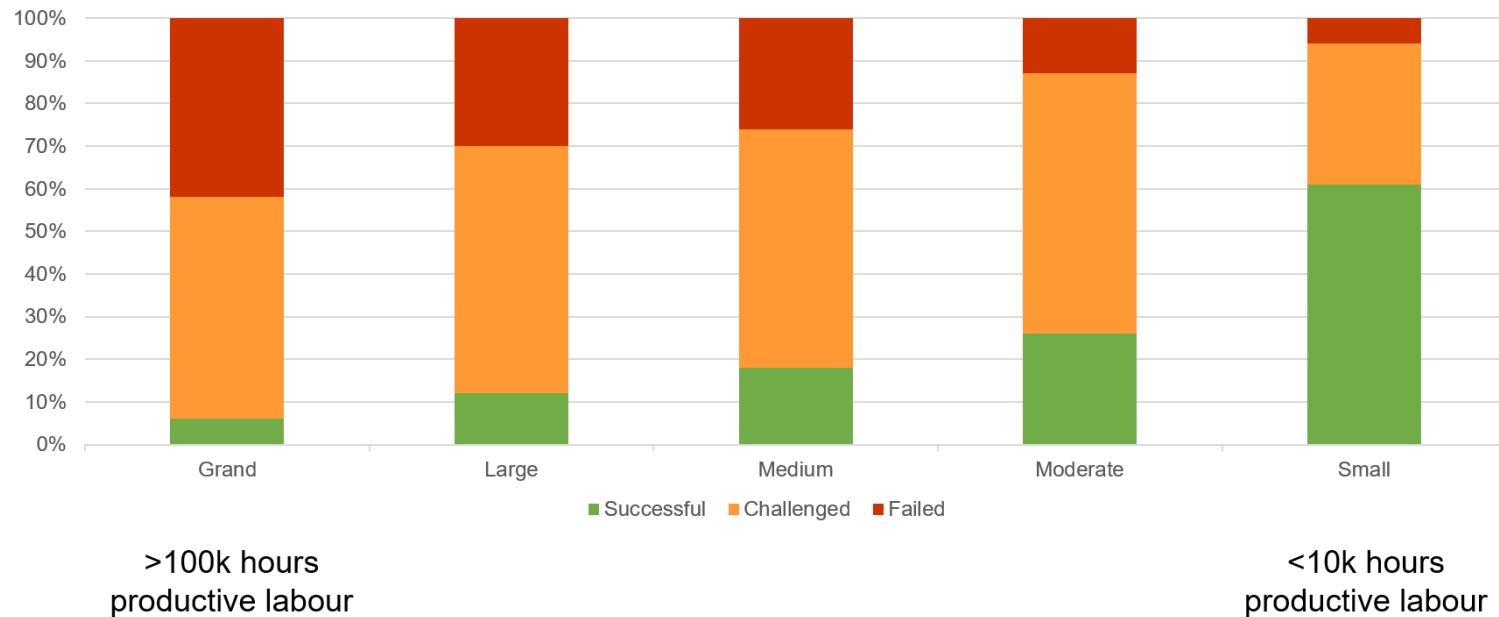
CHAOS Report 2020

- ▶ 2- Small Agile projects are more successful than large Waterfall ones



CHAOS Report 2020

- 2bis- small projects are more successful than large projects



CHAOS Report 2020

- ▶ 3- What Doesn't Predict Success
 - ▶ Project success isn't driven by a PM's "technical PM" skills or rigid methodology adherence; it comes from soft skills that build effective teams, fast decision-making, and clear direction.
 - ▶ Rolling out new tools doesn't improve outcomes by itself; success grows when teams are empowered to choose fit-for-purpose tools and unblock delivery.
 - ▶ Freezing objectives and requirements too early backfires; iterative discovery and evolving clarity lead to better, more valuable solutions.

Software Product Failure

- ▶ Software failure includes many types of problems
 - ▶ This is one of the types of project failures
- ▶ Capers Jones studied failures and the origins of those bugs (1996)
 - ▶ Code errors : 38.33%
 - ▶ Design errors : 24.17%
 - ▶ Documentation errors : 13.33%
 - ▶ Requirements errors : 12.50%
 - ▶ Bad-fix errors : 11.67%
- ▶ More errors are generated by coding

Software Product Failure (2)

- ▶ Errors introduced during the requirements phase
 - ▶ can propagate into design and coding
 - ▶ may turn into several design and coding problems
 - ▶ may not be discovered until after the product release
 - ▶ are generally more expensive than fixing a coding error to be fixed.
- ▶ The problem is that
 - ▶ Requirement and design specifications are not directly executable
 - ▶ A prototype should be built for test them (it is convenient?)
 - ▶ Typically errors are detected through reviews and formal inspections (high cost)

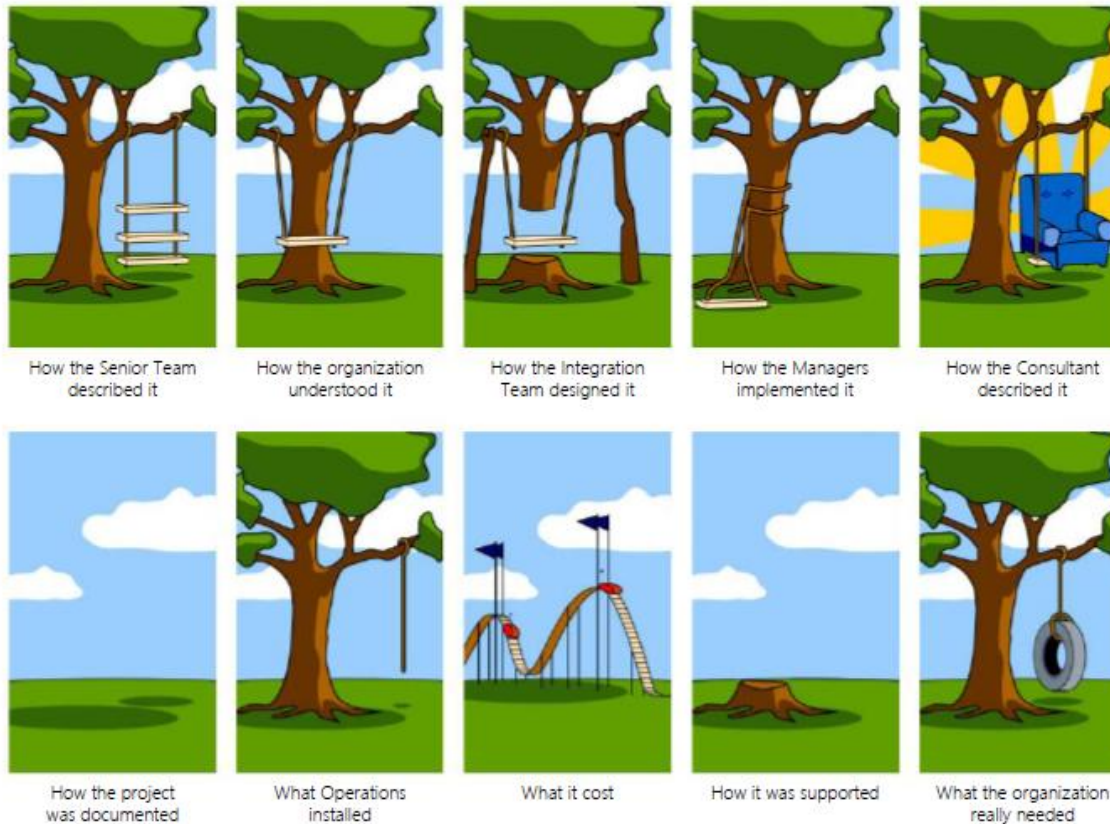
Coordination and other concerns

- ▶ Many failures are attribute to bad software, but
 - ▶ “As systems grow more complicated, failures have far less technical explanations: bad management, communications or training”
Association Press Report
- ▶ To reduce risks, many corporations are buying established enterprise software products (SAP, Oracle, ...)
 - ▶ Using consultants and outsourcing the implementation / customization of the systems

Software Engineering



Software Perspectives



Software development and its diverse stakeholder structure.

Software Engineering

- ▶ The term was first introduced in a 1968 NATO conference
 - ▶ During the 60's the computing field was starting to face a software crisis
 - ▶ Hardware increased his computing power and storage capacity
 - ▶ The first large and complex software applications requiring a large number of programmers have been required
 - ▶ Early experience with writing programs proved to be insufficient
 - ▶ Many projects were delayed, other never completed.
 - ▶ Low quality
- ▶ Need for a discipline much like the other engineering disciplines
 - ▶ The concept of software engineering has been formalized

Definitions of Software Engineer

- ▶ Software Engineer is a multiperson construction of multiversion software (Parnas 2001)
- ▶ An engineering discipline whose focus is the cost-effective development of high quality software system (Sommerville 2004)
 - ▶ He further explains that software is abstract and intangible
 - ▶ Refined definition:
“An engineering discipline that is concerned with all aspects of software production, from the early stages of system specification to maintaining the system after it has gone into use”
 - ▶ The definition captures the software life cycle.
- ▶ In computing we can focus on the “computers and programming languages themselves or we can view them as tools to be used in designing and implementing a solution to a problem. Software Engineering take the latter view” (Pfleeger 2001)

Definitions of Software Engineer (2)

- ▶ The Software Engineering curriculum Guidelines for Undergraduate Degree Programs in Software Engineer published by ACM and IEEE highlight 3 definitions:
 - ▶ Bauer: The establishment and use of sound engineering principles (methods) in order to obtain economically software that is reliable and works on real machines
 - ▶ CMU/SEI-90-TR-003 - form of engineering that applies the principles of computer science and mathematics to achieving cost-effective solutions to software problems (Software Engineering Institute, Carnegie Mellon University)
 - ▶ IEEE std 610-1990 - “application of a systematic, disciplined, quantifiable approach to the i) development of, ii) operation of, and iii) maintenance of software”

Definitions of Software Engineer (3)

- ▶ From the Curriculum Guidelines

Software Engineering is about creating high-quality software in a systematic, controlled and efficient manner. Consequently, there are important emphases on analysis and evaluation, specification, design and evolution of software. In addition, there are issues related to management and quality, to novelty and creativity, to standards, to individual skills, and to teamwork and professional practice that play a vital role in software engineering

- ▶ Capturing Software Engineering in a single sentence is difficult due to its breadth and short history

Definitions of Software Engineer (4)

- ▶ Software Engineering is a broad field that touches upon all aspects of a) developing and b) supporting a software system, spanning across the following key areas:
 1. Technical and business processes
 2. Specific methodologies and techniques
 3. Product characterization and metrics for measurements
 4. People skills and team work
 5. Tools and training
 6. Project coordination and management

Software Engineering Profession and Ethics

- ▶ There is no equivalent “professional engineer” (PE) designation for software engineers, yet in most of the countries.
- ▶ In Italy, this qualification exists but ...
 - ▶ The qualification is not needed for performing business work

Software Engineering Code of Ethics

- ▶ 8 principles for ethics and professional practices in software engineering
 - 1. PUBLIC
 - 2. CLIENT AND EMPLOYER
 - 3. PRODUCT
 - 4. JUDGMENT
 - 5. MANAGEMENT
 - 6. PROFESSION
 - 7. COLLEAGUES
 - 8. SELF

<https://ethics.acm.org/code-of-ethics/software-engineering-code/> IEEE-CS / ACM Version 5.2



Software Engineering Code of Ethics and Professional Practice

► PREAMBLE

- Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:
 1. PUBLIC - Software engineers shall act consistently with the public interest.
 2. CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
 3. PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
 4. JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.
 5. MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
 6. PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
 7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.
 8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Professional Behavior

- ▶ Some simplistic summarized “guidelines”
 - ▶ Respect others
 - ▶ Strive for fairness
 - ▶ Perform to one’s best capability
 - ▶ Follow the law
- ▶ There are plenty of opportunities where these guidelines would apply
 - ▶ Handling information privacy
 - ▶ Handling quality issues and problem resolutions
 - ▶ Handling project estimation and project coordination
 - ▶ Handling reuse and intellectual property
 - ▶ Handling security

Principles of Software Engineering

Alan Davis (1994)

- ▶ This are the rules and assumptions in software engineering derived from extensive observations
- ▶ Davis principles
 1. Make quality number 1
 2. High quality software is possible
 3. Give products to customers early
 4. Determine the problem before writing the requirements
 5. Evaluate design alternatives
 6. Use an appropriate process model

Principles of Software Engineering (2)

Alan Davis (1994)

7. Use different languages for different phases
8. Minimize intellectual distances
9. Put techniques before tools
10. Get it right before you make it faster
11. Inspect code
12. Good management is more important than good technology
13. People are the key to success
14. Follow with care
15. Take responsibility

Wasserman's Fundamental Software Engineering Concepts

- ▶ Anthony Wasserman (1996) suggested eight basic concepts thus form a foundation to the software engineering discipline:
 - ▶ **Abstraction** is a technique that is often categorized as a design technique. Software engineers aids in generalization of a concept and postponing the concerns for the details.
 - ▶ **Analysis and design methods and notation.** Analysis provides us a clear set of requirements, and design transforms the requirements to the solution structure. Many have attempted to omit or reduce these efforts, and such incomplete work has been the major cause of many software failures. In addition, the notation needs to be standardized to facilitate more universal communications. Developing standard notation is crucial in order for software engineering to move toward a disciplined engineering profession.
 - ▶ **User interface prototyping** contributes to a vital portion of understanding requirements and user preferences. It also contributes heavily to an iterative process approach to software development
 - ▶ **Modularity and architecture** contributes heavily to the quality and maintainability of the software
 - ▶ **Reuse** is a long-standing practice in the software industry.
 - ▶ **Life cycle** and process has to be managed.
 - ▶ **Metrics process.** Using metrics is an engineering technique that allows to provide an assess to the software.
 - ▶ **Tools and integrated environment** are necessary for the development and support of any large, complex software.

Software Process



Software Process

- ▶ The process of developing and supporting software requires many distinct tasks to be performed by different people in some related sequence
 - ▶ If software engineers are left to perform tasks based on their experience, knowledge and background, they do not perceive and perform the tasks in the same way / order
 - ▶ This inconsistency may cause poor end products and / or project failures

Goal of the Software Process Models

- ▶ The goal is to provide guidance for systematically coordinating and controlling the tasks to be performed for achieving the end product and the project objectives
- ▶ It defines:
 - ▶ A set of tasks that need to be performed
 - ▶ The input to and the output from each task
 - ▶ The preconditions and post-conditions for each task
 - ▶ The sequence and flow of these tasks
- ▶ If it is considered as a roadmap for generating various intermediate deliverables (e.g., design document, user guide, ...), the Software Process Model is needed also when only one person is involved in software development

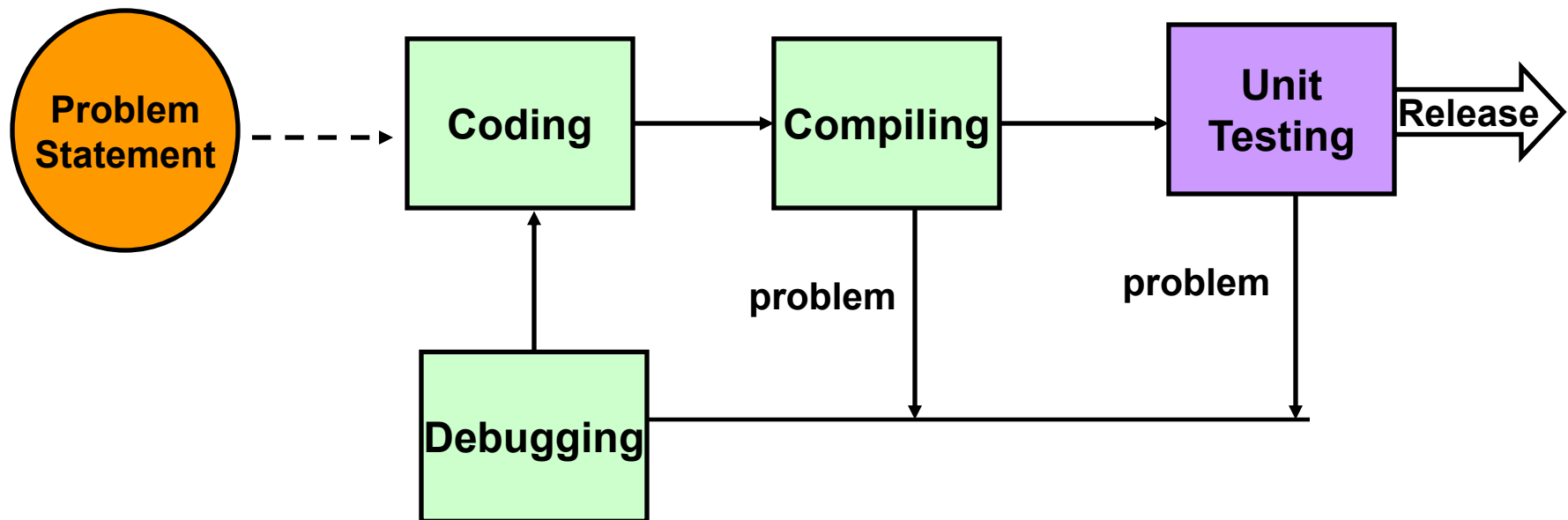
Goal of the Software Process Models

- ▶ For complex projects involving multiple people, having a defined process will provide some of the following benefits:
 - ▶ Improved understanding of the tasks to be performed by the team members
 - ▶ Better defined responsibilities, expectations, and results from the project team members
 - ▶ Easier measurement and control of the project progress

The “Simplest” Process Model

- ▶ When programmers are left alone, they naturally gravitate to what they perceive as the most important task,
 - ▶ This is ... coding!
- ▶ All people involved in IT usually starts the career to write code in some programming language
- ▶ In this model the code is considered the central task
 - ▶ The model is called “code and fix”

The “Simplest” Process Model (2)



- ▶ Problem statement = requirements specifications
- ▶ Unit testing = performed in some informal way by the code writer
 - ▶ Most people performs and follow this simple process, but unfortunately some skips unit testing or debugging.

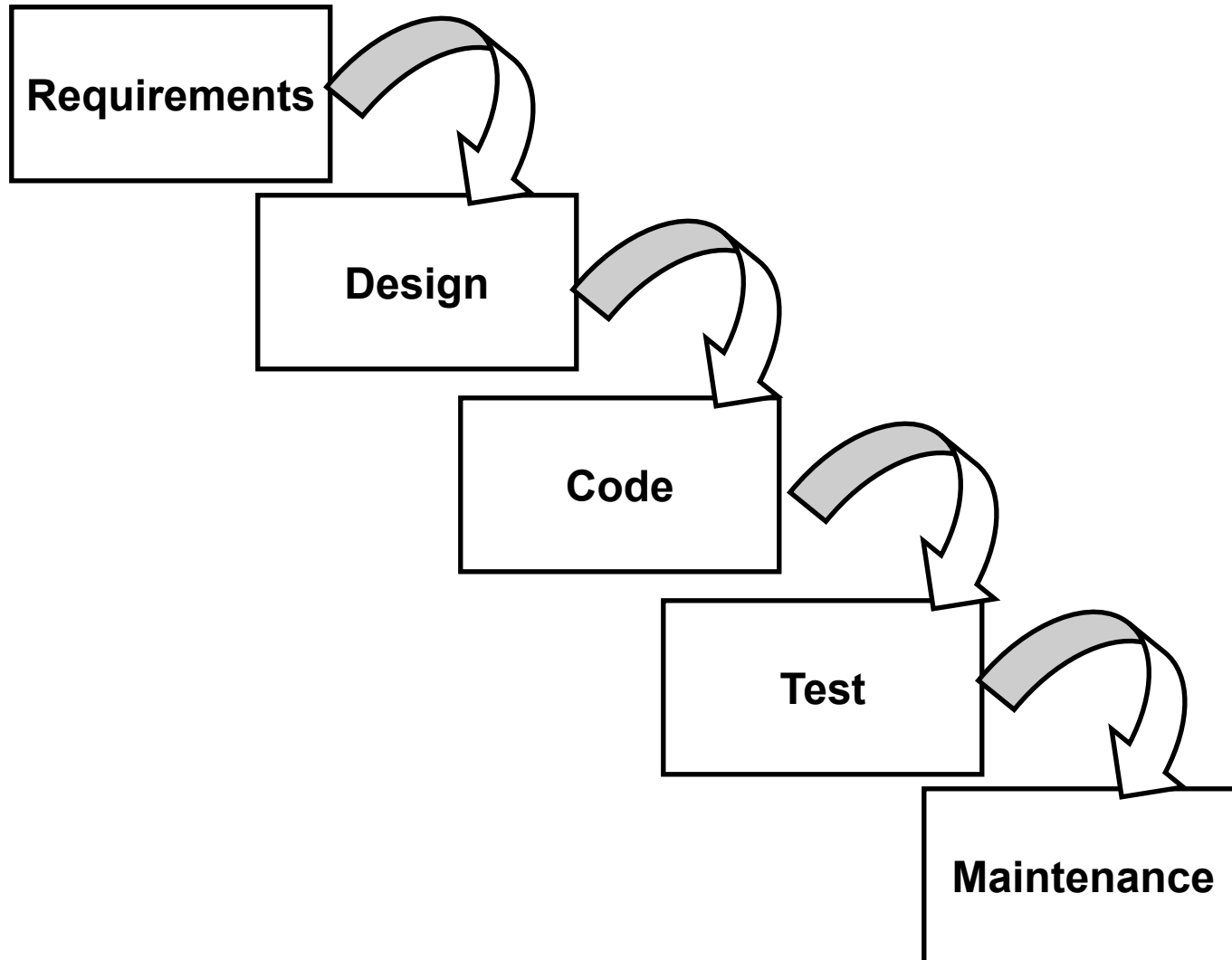
Extending the “Simplest” Process Model

- ▶ As projects got larger and more complex.
 - ▶ Need to clarify and stabilize the requirements
 - ▶ Need to test more functionalities
 - ▶ Need to design more carefully
 - ▶ Need to use more existing software & tools
 - ▶ Database
 - ▶ Network
 - ▶ Code control
 - ▶ Need more people to be involved

Extending the “Simplest” Process Model (2)

- ▶ Waterfall : earliest process and coping with no process
 - ▶ Incremental : coping with decomposing the large systems
 - ▶ Spiral : coping with risk management
 - ▶ Rational Unified Process : coping with different task and managing through project phases
 - ▶ ... and many other ones
-
- ▶ To be introduced in the "Progettazione del Software course in the Master Degree

Waterfall Model



Waterfall Model (2)

- ▶ Royce (1970): tasks occur sequentially one after another with the output from one task dropping into the next task
 - ▶ It serves as a tool for managing software projects
 - ▶ It represents the software life cycle as the software goes through different and distinct stages of development
 - ▶ It gives the project managers a way to describe the status of a project
- ▶ Positive aspects
 - ▶ Requirements must be specified in the first step
 - ▶ 4 main tasks must be completed before the system can be packaged for release (requirements, design, code and test)
 - ▶ The output from each stage is fed into the next stage in sequence
 - ▶ The project can be tracked and moves sequentially through identifiable tasks

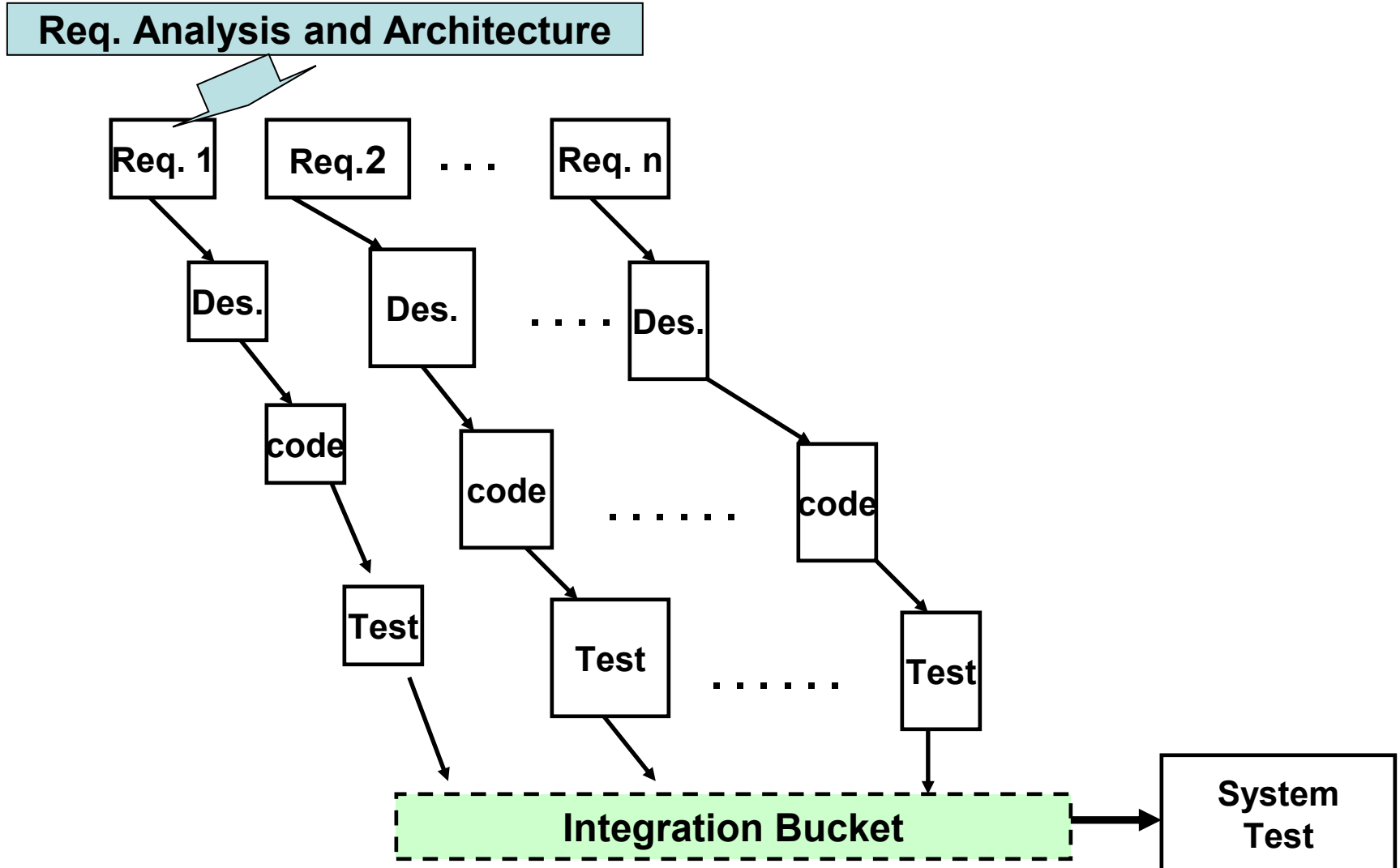
Waterfall Model (3)

- ▶ It has also called document-driven approach for the amount of documents generated in each stage
- ▶ Many modification of the model have been proposed
 - ▶ Backward arrows introduced to depict the addition of iterative activities
- ▶ Low interaction with the users (only in the requirements stage and at the delivery)

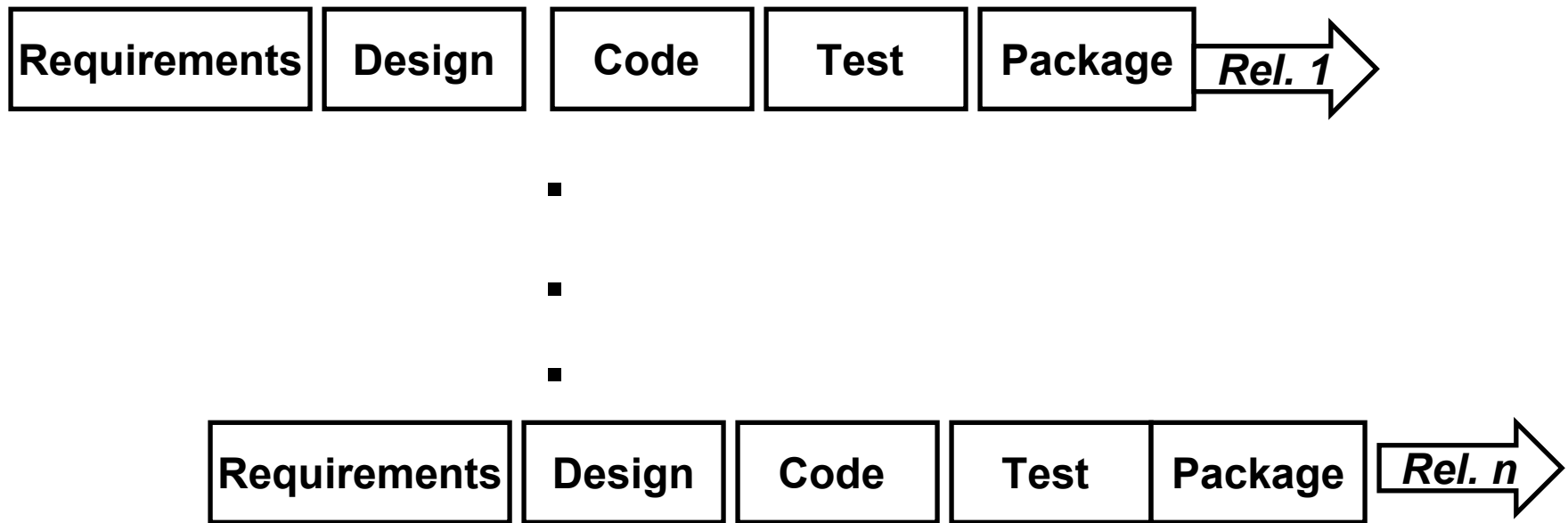
Incremental Model - Waterfall

- ▶ It can be viewed as a modification to the waterfall model
- ▶ Large projects are much easier if subdivided into smaller components
 - ▶ The components may be developed incrementally and iteratively
 - ▶ The components are then integrated and test as a whole in a final system test
- ▶ It implements a sort of risk containments
 - ▶ If any component has a problem, the other components are able to be developed independently
- ▶ It allows to develop firstly the core components
 - ▶ Additional features can be released after

Multiple-Component Incremental Model



Multiple-Release Incremental Model



- ▶ Each small set of requirements is developed, packages, and released in a multiple release fashion.
- ▶ Release 1 is the core function followed by subsequent releases including
 - ▶ Fixes of bugs
 - ▶ New functional features

Incremental Model

- ▶ Waterfall and incremental models implements the “divide and conquer” methodology
 - ▶ Typically problems are intertwined making the decoupling of the parts into independent component difficult
 - ▶ It requires a deep understanding of the problem, solution and usage environment
- ▶ The multiple release is a representation of and a precursor of today’s notion of continuous integration and continuous deployment, CI/CD.
 - ▶ The key in CI/CD is the “continuous” integration of completed parts and then delivering that to the users for quick deployment.
 - ▶ A good build and integration tool is a necessity for CI/CD.



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Fifth Edition), Jones & Bartlett Learning

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.